

Foundry → Databricks Migration Project

Full Project Documentation

UDH 3.0 Platform · 50 Datasets · 12 Weeks · AWS + Databricks + MARS

Generated: 30 April 2026

50	12 Datasets to migrate	3 Weeks duration	2 AI roles in project
----	---------------------------	---------------------	--------------------------

Table of Contents

1.	Project Overview	3
2.	Architecture — End to End	4
3.	Technology Stack Explained	5
4.	What is Medallion Architecture	6
5.	Palantir Foundry vs Databricks	7
6.	MARS Platform — AI at UAL	8
7.	Your Role — Code Migration	9
8.	Code Conversion — How It Works	10
9.	Code Conversion Application — Spec	11
1		
0.	RAG Application — What We Built	13
1		
1.	AI Roles in the Project	14
1		
2.	SOW Summary — Key Commitments	15
1		
3.	Building with Claude CLI — Guide	16

1. Project Overview

Background

United Airlines (UAL) is migrating its data processing infrastructure from Palantir Foundry to the United Data Hub (UDH) 3.0 platform, which runs on AWS Databricks. The objective is to modernize datasets for AirOps and Corporate & Digital Portfolio divisions, achieving lower costs, open architecture, and tighter integration with UAL's MARS AI platform.

Project Statement

Migrate and modernize 50 datasets from Palantir Foundry to Databricks on AWS, implementing Medallion Architecture (Bronze/Silver/Gold) using Delta Lake on S3, with full governance via the ICON framework.

Project At a Glance

Parameter	Detail
Client	United Airlines (UAL)
Duration	12 Weeks — March 16 to June 5, 2026
Scope	50 datasets — AirOps + Corporate & Digital Portfolio
Source Platform	Palantir Foundry
Target Platform	Databricks on AWS (UDH 3.0)
Storage Format	Delta Lake on AWS S3 (Iceberg format)
Architecture	Medallion — Bronze → Silver → Gold
Orchestration	Apache Airflow and/or Databricks Workflows
Governance	ICON Framework (Ingest, Catalog, Optimize, Notify)
AI Platform	MARS — AWS Bedrock
Validation	UAL Test Harness framework
Delivery	Dev → Stage → Production environments

2. Architecture — End to End

High Level Data Flow

The complete UAL data pipeline flows from legacy source systems through Foundry/Airflow ingestion into Databricks for transformation, then stores cleaned data in S3 Delta Lake across three quality layers, before serving analytics via Redshift and powering AI models via MARS.

1. Data Sources	SFTP, Teradata, Oracle	Raw operational data from legacy source systems
2. Ingestion	Palantir Foundry (legacy) Apache Airflow	Foundry is being replaced. Airflow stays as orchestrator for DAGs
3. Compute	Databricks (PySpark)	Runs all transformation logic — the new engine replacing Foundry
4. Bronze Layer	AWS S3 + Delta Lake	Raw data landed exactly as-is from source. No transformations
5. Silver Layer	AWS S3 + Delta Lake	Cleaned, deduplicated, validated, standardised data
6. Gold Layer	AWS S3 + Delta Lake	Business-ready aggregated data, optimised for analytics
7. Serving	AWS Redshift + ICON	Redshift for SQL analytics. ICON for governance and metadata
8. AI / ML	MARS (AWS Bedrock)	UAL's AI platform — consumes Gold data to run production AI models
9. Consumers	BI Tools, Employees, Customers	Dashboards, internal tools, customer-facing AI notifications

Migration Scope

The 12-week project covers layers 2–7 above — everything from Ingestion through to Serving. MARS (layer 8) and Consumers (layer 9) are already running in production and will automatically benefit from cleaner Gold data.

3. Technology Stack Explained

Palantir Foundry

Proprietary enterprise data platform by Palantir Technologies. Being replaced at UAL due to high cost and vendor lock-in. Contains all existing Python transform code to be migrated.

Apache Airflow

Open-source workflow orchestration tool. Defines DAGs (Directed Acyclic Graphs) that control the order and scheduling of pipeline tasks. Stays in the new architecture.

Databricks

Cloud data and AI platform built on Apache Spark. UAL's new compute engine. Engineers write Python/SQL notebooks that process data at scale. Runs natively on AWS.

AWS S3

Amazon Simple Storage Service. Stores all data files (Bronze/Silver/Gold). Cheap, durable, infinitely scalable object storage. The lakehouse foundation.

Delta Lake

Open-source storage layer that adds ACID transactions, time travel, and schema enforcement on top of S3. Data is stored as Parquet files + a transaction log.

AWS Redshift

Cloud data warehouse for fast SQL analytics. BI tools connect here. Optimised for analytical queries on large datasets.

ICON Framework

UAL's internal governance framework — Ingest, Catalog, Optimize, Notify. Handles metadata capture, data quality validation, and governance enforcement.

MARS

UAL's internal AI/ML platform built on AWS Bedrock. Runs production LLM models like 'Every Flight Has a Story'. Consumes Gold layer data as features.

AWS Bedrock

Amazon's managed AI service. Provides access to foundation models from multiple vendors. MARS is built on top of Bedrock.

4. What is Medallion Architecture

Medallion Architecture is a data design pattern that organises data into three progressive quality layers. Same data, getting cleaner and more refined at each stage.

Bronze Layer — Raw Zone

Data lands exactly as it came from the source. No changes, no cleaning. Serves as an audit trail and reprocessing safety net.

Example: A CSV from SFTP dropped in as-is, with nulls, duplicates, messy date formats

Silver Layer — Cleaned Zone

Data is cleaned, validated, and standardised. Duplicates removed, nulls handled, data types fixed, joins applied. Trusted data for engineers and analysts.

Example: Same CSV — nulls filled, dates normalised to ISO format, duplicate rows removed

Gold Layer — Business Zone

Data is aggregated and modelled for specific business use cases. Pre-joined, pre-calculated, business-friendly naming. Ready for BI tools and MARS.

Example: Total revenue by region per month — already calculated and stored, zero compute at query time

Storage Format: Delta Lake on S3

All three layers use Delta Lake format — not plain Parquet. Delta adds ACID transactions, time travel (query historical versions), schema evolution, and upsert/delete support on top of standard Parquet files stored in S3.

Feature	Delta Lake Support
Format	Delta (Parquet files + <code>_delta_log</code> transaction log)
ACID	Yes — guaranteed consistency
Time Travel	Yes — query any historical version
Upserts	Yes — merge/update records
Schema Evolution	Yes — add columns without breaking downstream
Location	AWS S3 buckets

5. Palantir Foundry vs Databricks

Simple Analogy

Restaurant vs Kitchen

Foundry = A full restaurant. Everything provided — kitchen, waiters, menu, décor. Easy for anyone. But expensive and you cannot change how the kitchen works. Databricks = A professional kitchen. Maximum power and control. But you need to be a chef (data engineer) to use it. Pay only for what you use.

Side by Side Comparison

Aspect	Palantir Foundry	Databricks
Made by	Palantir Technologies (independent co.)	Databricks Inc. (independent co.)
Cloud	Runs on AWS / Azure / GCP	Runs on AWS / Azure / GCP
Who uses it	Business analysts + non-technical teams	Data engineers + scientists
How you work	Click, drag and drop — visual UI	Write Python / SQL code
Cost	Expensive fixed enterprise license	Pay per use — consumption based
Ownership	Locked in — Palantir controls everything	Open — Apache Spark foundation
Code portability	Hard to extract — proprietary format	Standard Python / SQL — portable
AWS integration	Works but external to AWS ecosystem	Native — built for AWS
ML support	AIP for AI workflows	MLflow + native ML lifecycle
UAL status	Being replaced (this project)	New platform (this project)

Why UAL is Making This Change

- **Cost reduction:** Foundry enterprise licensing is significantly more expensive than Databricks' consumption model
- **Open architecture:** Databricks is built on Apache Spark (open source) — no vendor lock-in
- **AWS-native:** UAL is fully on AWS. Databricks is native to AWS; Foundry sits outside the ecosystem
- **Engineer productivity:** UAL's data team writes Python/SQL. Databricks is their natural environment
- **MARS integration:** Databricks Gold layer feeds directly into MARS on AWS Bedrock — tighter integration
- **Control:** UAL owns and manages the platform vs renting Foundry from Palantir

6. MARS Platform — AI at United Airlines

What is MARS?

MARS is United Airlines' internal AI and Machine Learning platform, built on AWS Bedrock. It sits on top of the United Data Hub (UDH) and consumes the Gold layer data produced by the Databricks pipeline.

Attribute	Details
Platform name	MARS (Machine learning and AI platform)
Built on	AWS Bedrock
Foundation	United Data Hub (UDH)
Purpose	Run AI/ML models in production for employees and customers
Key model	'Every Flight Has a Story' — LLM for flight notifications
Impact	6% improvement in overall customer satisfaction
Notifications	100,000+ push notifications sent to passengers
Future use cases	90+ additional applications identified

How Your Migration Feeds MARS

The data you migrate and clean through the Bronze → Silver → Gold pipeline becomes the training data, feature store, and inference input for MARS models. Cleaner, more reliable Gold data means better AI models.

Your Work	Impact on MARS
Your work	Impact on MARS
50 datasets migrated to Databricks	Feeds richer, more complete data to MARS
Bronze → Silver cleaning	Removes noise from ML training data
Gold layer aggregations	Pre-built features for faster model inference
ICON governance	Ensures data quality MARS models depend on
Automation utility	Could auto-onboard new ML datasets into MARS

7. Your Role — Code Migration

Core Responsibility

Your Job in One Sentence

Read every Foundry Python transform file, understand the logic exactly, and rewrite it as an equivalent Databricks notebook — with zero changes to business logic.

The Golden Rule

Everything should be same only. No optimisation. No restructuring. No simplification. The output from Databricks must match the output from Foundry exactly — same row counts, same column values, same logic. If anything is missed the business will raise issues.

Migration Wave Plan

Timeline	Phase	Key Activities
Wk 1–4	Analysis & Design	Inventory, classify, map lineage for all 50 datasets
Wk 4–5	Build Wave 1	Convert first 20 datasets to Databricks notebooks
Wk 6	SIT Wave 1	System Integration Testing — validate all 20 datasets
Wk 7	UAT Wave 1	Business user validation for Wave 1
Wk 7–8	Build Wave 2	Convert next 23 datasets
Wk 9	SIT Wave 2	System Integration Testing for Wave 2
Wk 10–11	UAT Wave 2	Business user validation for Wave 2
Wk 12	Cutover / Go Live	Final production deployment of all 50 datasets

Dataset Complexity Distribution

	Count	Definition
	20% = 10 datasets	Window functions, merges, complex joins
	40% = 20 datasets	Multiple joins, aggregations, cleaning

	40% = 20 datasets	Basic filter, select, rename operations
--	-------------------	---

8. Code Conversion — How It Works

Foundry to Databricks API Mapping

Foundry API	Databricks Equivalent
@transform decorator	Plain Python function — no decorator needed
Input('/path')	spark.read.format('delta').load('s3://...')
Output('/path')	df.write.format('delta').mode('overwrite').save('s3://...')
input.dataframe()	spark.read.format('delta').load(path)
output.write_dataframe(df)	df.write.format('delta').mode('overwrite').save(path)
dataframe('previous')	Delta time travel: .option('versionAsOf', n).load(path)
ctx.get_parameter('x')	dbutils.widgets.get('x')
Data Lineage graph	Databricks Workflows with task dependencies
Python Transform function	Databricks notebook cell with %python magic
Uploaded Dataset	S3 path or DBFS path reference

Conversion Example

Before (Foundry):

```
# FOUNDRY CODE
from transforms.api import transform, Input, Output
from pyspark.sql import functions as F

@transform(
    output=Output("/UAL/cleaned/case"),
    case_input=Input("/UAL/raw/case"),
)
def cleaned_case(output, case_input, ctx):
    df = case_input.dataframe()
    df = df.filter(F.col("closed_date").isNotNull())
    df = df.dropDuplicates(["case_id"])
    output.write_dataframe(df)
```

After (Databricks):

```
# DATABRICKS NOTEBOOK
# COMMAND -----
# %md ## Cleaned Case – Silver Layer

# COMMAND -----
from pyspark.sql import functions as F
```

```
# Parameters
S3_INPUT  = dbutils.widgets.get("S3_INPUT_PATH")
S3_OUTPUT = dbutils.widgets.get("S3_OUTPUT_PATH")

# Read
df = spark.read.format("delta").load(S3_INPUT)

# Transform (same logic – no changes)
df = df.filter(F.col("closed_date").isNotNull())
df = df.dropDuplicates(["case_id"])

# Write
df.write.format("delta").mode("overwrite").save(S3_OUTPUT)

# COMMAND -----
# Validation
print(f"Output rows: {df.count()}")
```

9. Code Conversion Application — Specification

Why We Need a Dedicated Application

Converting a single file in isolation misses cross-file imports and shared utilities. The application must understand ALL files in a project together — resolving imports, shared configs, and utility functions — before converting any single file.

Key Insight

If `cleaned_case.py` imports from `utils/common.py` and `utils/date_utils.py`, converting `cleaned_case.py` alone will produce broken code. The app must index ALL files, resolve ALL imports, then convert with full context.

Application Modules

1. Project Manager

Create and manage separate projects (UAL-AirOps, UAL-Corporate, etc.). Each project has its own isolated RAG store — no mixing between projects.

- Create / rename / delete projects
- Switch between projects
- View project stats (files, chunks, conversion status)

2. File Uploader

Accept files in multiple formats and auto-process them.

- Upload single .py file
- Upload multiple .py files at once
- Upload ZIP folder — auto extract all .py files
- Auto-detect folder structure and import paths
- Show upload progress and indexing status

3. RAG Engine (per project)

Index all files and build a knowledge graph of the entire project.

- Chunk all files into 50-line blocks
- Map all import statements across files
- Build dependency graph (which file imports which)
- Store in isolated per-project vector/keyword store
- Update index when new files are added

4. Converter

Convert individual or all files with full cross-file context.

- Select target file to convert
- Auto-find all imported files from RAG

- Send full context (file + all imports) to Claude AI
- Convert with complete picture — no missing dependencies
- Apply Foundry → Databricks API mapping
- Preserve all business logic exactly
- Mark each converted file as done

5. Output Manager

Manage and download converted files.

- View converted notebook side by side with original
- Highlight unconverted Foundry APIs (validation)
- Download single converted file
- Download all converted files
- Download as ZIP with original folder structure
- Show conversion status per file (Pending / Done / Error)

Input File Types Supported

Input Type	Description
Single .py file	Paste or upload one Foundry transform
Multiple .py files	Drag and drop several files at once
ZIP archive	Auto-extracted — preserves folder structure
Folder path	Future — direct folder reference
Git repository	Future — clone and index entire repo

How Cross-File Import Resolution Works

Project files uploaded:

```

■■■ cleaned_case.py           ← wants to convert this
■■■ cleaned_supervisor.py
■■■ utils/common.py           ← imported by cleaned_case.py
■■■ utils/date_utils.py       ← imported by cleaned_case.py
■■■ utils/constants.py

```

Step 1: RAG indexes ALL 5 files

Step 2: User selects cleaned_case.py to convert

Step 3: App parses: "from utils.common import get_config"

Step 4: App retrieves utils/common.py from RAG

Step 5: App parses: "from utils.date_utils import parse_date"

Step 6: App retrieves utils/date_utils.py from RAG

Step 7: Sends cleaned_case.py + common.py + date_utils.py to Claude

Step 8: Claude converts ALL with full context

Step 9: Output: converted cleaned_case.py with correct Databricks imports

10. RAG Application — What We Built

Current Application (v2)

We have already built a working RAG-powered Code Intelligence application as a React artifact. This serves as the foundation for the full conversion app.

Feature	Description
Architecture tab	Full UAL pipeline diagram — Sources → Foundry → Databricks → Storage → MARS → Consumers
Ask AI tab	Upload .py files, ask questions in plain English about the code logic
Code Viewer tab	Browse all uploaded files with syntax highlighting and complexity rating
Generate tab	Select file + layer (Bronze/Silver/Gold) → auto-generate Databricks notebook
Complexity classifier	Automatically rates each file Simple / Medium / Complex based on code analysis
RAG engine	Chunks files into 50-line blocks, scores by keyword relevance, sends top chunks to Claude
Project-aware system	Claude understands UAL architecture, ICON, Airflow, Delta Lake context

How RAG Works in the App

Step	Action
Step 1 — Upload	User uploads .py files from Foundry
Step 2 — Chunk	Each file split into 50-line blocks
Step 3 — Index	All chunks stored in memory with filename + line numbers
Step 4 — Query	User asks a question
Step 5 — Score	Each chunk scored by keyword match to the question
Step 6 — Retrieve	Top 5 most relevant chunks selected
Step 7 — Augment	Chunks sent to Claude as context
Step 8 — Generate	Claude answers based on actual code, not general knowledge

What to Upgrade Next

Feature	Purpose
Project isolation	Separate RAG store per project
ZIP upload	Auto-extract and index entire project
Import resolution	Parse and follow cross-file imports
Bulk conversion	Convert all files in one click
Download output	Export converted files as ZIP
Conversion tracker	Status per file — Pending / Done / Error
Side by side diff	Original Foundry vs converted Databricks
Validation check	Flag any remaining unconverted Foundry APIs

11. AI Roles in the Project

There are three distinct AI roles in this project:

AI Role 1 — Helping Your Team (Now)

The RAG app we built. Claude AI reads the Foundry code and helps engineers understand and convert it. Saves hours of manual code reading per dataset × 50 datasets.

Internal tool · Used during 12-week migration · Your team only

AI Role 2 — Automation Utility (SOW Deliverable)

AI that auto-generates Databricks notebooks from metadata config files. Engineers define what they want in a config — AI writes the notebook. Also auto-creates Airflow DAGs and auto-ingests to Bronze layer.

SOW deliverable · Ready by Week 8 · Engineers use it

AI Role 3 — MARS Production AI (Always Running)

UAL's production AI platform on AWS Bedrock. Runs LLMs for customer notifications, delay communications, and employee tools. Consumes Gold layer data your migration produces.

Production system · Millions of users · Your clean data feeds it

12. SOW Summary — Key Commitments

Scope of Work

- Migrate 50 datasets from Foundry to Databricks on AWS (UDH 3.0)
- Implement Medallion Architecture (Bronze/Silver/Gold) with Delta Lake
- Design and orchestrate pipelines using Apache Airflow and/or Databricks Workflows
- Create metadata artifacts and coordinate with UAL SMEs and ICON Governance team
- Leverage ICON framework for metadata capture, data quality, and governance
- Implement Foundry-UDH synchronisation via Foundry Virtual Table mechanism
- Use UAL Test Harness for data validation and reconciliation
- Ensure all pipelines are in Disaster Recovery (DR) workflows
- Deploy artifacts across Stage and Production environments
- Build AI-led Automation Utility for data onboarding
- Conduct Knowledge Transfer (KT) sessions and code handover to DevOps team
- Execute final production cutover and Go-Live support

Success Criteria

- All 50 datasets available in UDH or Foundry for consumption
- Zero data loss or transformation mismatch during migration
- Job performance and data delivery SLAs met or improved in Databricks
- All required metadata and data quality processes in place using ICON
- Project delivered within 12 weeks with formal milestone approvals
- Working demo for Automation-led Data Onboarding Utility

Out of Scope

- Moving data from Databricks Delta to RDS or Redshift
- Provisioning infrastructure components (Databricks, AWS, Redshift)
- Procurement or licensing of third-party tools
- Long-term production support beyond hypercare period

13. Building with Claude CLI — Guide

What to Build

Target Application

A full-stack Code Conversion Application with project isolation, ZIP upload, cross-file import resolution, bulk conversion, and downloadable output. Built as a React frontend + Claude API backend.

Recommended Tech Stack

Layer	Technology
Frontend	React (JSX) — single page application
AI Backend	Anthropic Claude API — claude-sonnet-4-20250514
RAG store	In-memory per project (JS Map / object)
File handling	FileReader API for .py files, JSZip for ZIP extraction
Styling	Tailwind CSS utility classes
Deployment	Claude Artifact or standalone React app

Claude CLI Prompts to Use

Start the app

```
"Build a React code conversion application for migrating Palantir Foundry Python transforms to Databricks notebooks. Include: project manager, ZIP upload with auto-extraction, per-project RAG with cross-file import resolution, bulk converter using Claude API, conversion status tracker, and ZIP download of outputs."
```

Add ZIP handling

```
"Add ZIP file upload support using JSZip library. Auto-extract all .py files, preserve folder structure, parse import statements to build dependency graph, store in project RAG."
```

Add cross-file resolution

```
"When converting a file, parse all import statements, look up imported files in the project RAG, include them as context when calling the Claude API for conversion."
```

Add validation

```
"After conversion, scan the output for any remaining Foundry API calls (Input(), Output(), @transform, dataframe(), write_dataframe()) and flag them as unconverted with line numbers."
```

Add download

"Add download functionality – single file download and bulk ZIP download of all converted files preserving the original folder structure."

Key Claude API Call Pattern for Conversion

```
const response = await fetch("https://api.anthropic.com/v1/messages", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    model: "claude-sonnet-4-20250514",
    max_tokens: 1000,
    system: `You are a Foundry → Databricks migration expert.
Convert the provided Foundry Python transform to a Databricks notebook.
Rules: preserve ALL business logic exactly, replace Foundry APIs with
Databricks equivalents, use Delta format on S3, add validation cell.`,
    messages: [{
      role: "user",
      content: `Convert this file: ${mainFile}

Related imports also provided for context:
${importedFiles.join("\n\n")}`

Target layer: ${selectedLayer}`
    }]
  })
});
```

Development Order

- Step 1 — Project manager UI (create, switch, list projects)
- Step 2 — Single .py file upload + RAG indexing
- Step 3 — Single file conversion using Claude API
- Step 4 — ZIP upload + auto extraction (JSZip)
- Step 5 — Import parsing + dependency graph
- Step 6 — Cross-file context in conversion API call
- Step 7 — Bulk conversion (all files in project)
- Step 8 — Conversion status tracker per file
- Step 9 — Validation (flag unconverted Foundry APIs)
- Step 10 — Download single + ZIP of all converted files

Start Here

Begin with: 'Build a React app with project management and single .py file upload that converts Foundry Python to Databricks notebooks using the Claude API.' Then add features one by one using the prompts above.